SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI -602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – APPLICATION BASED QUESTIONS

Course Code:	DSA0501
Course Title:	Query Processing using Data Analysis
CO Assessed:	CO1
Assessment:	Tool 2 – Application Based Questions
Weightage:	
Total Marks:	20 Marks

CO1 Statement

Student Name: T. Nivya Sri Reg. No.: 192424131

Section/Batch: 2024 Date of Submission: 27-07-26

Faculty In-charge: Dr. Veena Devi K

Submission Mode: Individual

Code of Conduct

I T. Nivya Sri (192424131) certify that this submission is my original work and have adhered to all academic integrity guidelines.

Signature:

Criteria	Marks
Understanding of the Problem	4
Application of Concepts	4
Program Design / Algorithm	4
Correctness of Solution & Output	4
Explanation / Interpretation of Results	4
Total	20

CO1 -AT2: Application Based Questions

Hotel Reservation Management (10 Marks)

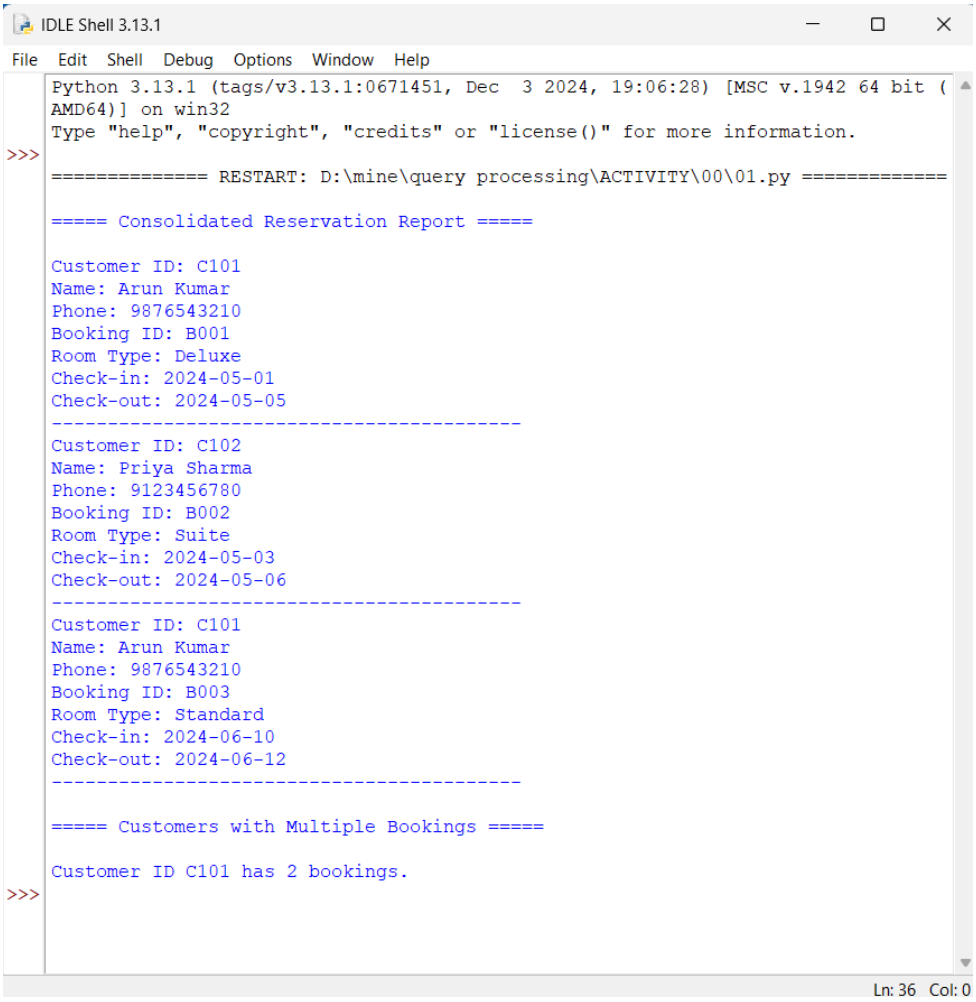
A hotel maintains booking details in CSV format and customer information in JSON format. Write a Python program to integrate the two datasets using the Customer ID, identify customers with multiple bookings, and generate a consolidated reservation report.

CODE:

```
import csv
import json
from collections import defaultdict
def load_bookings(csv_file):
    bookings = []
    with open(csv_file, mode='r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            bookings.append(row)
    return bookings
def load_customers(json_file):
    with open(json_file, 'r') as file:
        customers = json.load(file)
    return customers
def integrate_data(bookings, customers):
    consolidated = []
    for booking in bookings:
        cid = booking["CustomerID"]
        if cid in customers:
            merged = {**booking, **customers[cid]}
            consolidated.append(merged)
```

```
    return consolidated
def find_multiple_bookings(consolidated):
    count = defaultdict(int)
    for entry in consolidated:
        count[entry["CustomerID"]] += 1
    multiple = {cid: num for cid, num in count.items() if num > 1}
    return multiple
def generate_report(consolidated, multiple):
    print("\n===== Consolidated Reservation Report =====\n")
    for entry in consolidated:
        print(f"Customer ID: {entry['CustomerID']}")
        print(f"Name: {entry['Name']}")
        print(f"Phone: {entry['Phone']}")
        print(f"Booking ID: {entry['BookingID']}")
        print(f"Room Type: {entry['RoomType']}")
        print(f"Check-in: {entry['CheckIn']}")
        print(f"Check-out: {entry['CheckOut']}")
        print("----------------------------------------")
    print("\n===== Customers with Multiple Bookings =====\n")
    for cid, num in multiple.items():
        print(f"Customer ID {cid} has {num} bookings.")
bookings = load_bookings("bookings.csv")
customers = load_customers("customers.json")
consolidated = integrate_data(bookings, customers)
multiple = find_multiple_bookings(consolidated)
generate_report(consolidated, multiple)
```

OUTPUT:



Hospital Appointment Scheduling (10 Marks)

A hospital stores doctor information in JSON format and appointment records in CSV format. Write a Python program to merge both datasets, identify appointments assigned to unavailable doctors, and generate a report listing only valid appointments.

CODE:

```
import csv
import json
def load_doctors(json_file):
```

```
    with open(json_file, 'r') as file:
        doctors = json.load(file)
    return doctors
def load_appointments(csv_file):
    appointments = []
    with open(csv_file, 'r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            appointments.append(row)
    return appointments
def validate_appointments(appointments, doctors):
    valid = []
    invalid = []
    for appt in appointments:
        doc_id = appt["DoctorID"]
        if doc_id in doctors:
            doctor = doctors[doc_id]
            merged = {**appt, **doctor}
            if doctor["Available"]:
                valid.append(merged)
            else:
                invalid.append(merged)
        else:
            appt["Error"] = "Doctor not found"
            invalid.append(appt)
    return valid, invalid
def generate_report(valid, invalid):
```

```
    print("\n===== VALID APPOINTMENTS =====\n")
    for v in valid:
        print(f"Appointment ID: {v['AppointmentID']}")
        print(f"Patient: {v['PatientName']}")
        print(f"Doctor: {v['Name']} ({v['DoctorID']})")
        print(f"Date: {v['Date']}")
        print("----------------------------------")
    print("\n===== INVALID APPOINTMENTS (Unavailable/Unknown Doctors) =====\n")
    for iv in invalid:
        print(f"Appointment ID: {iv['AppointmentID']}")
        print(f"Patient: {iv['PatientName']}")
        print(f"Doctor ID: {iv['DoctorID']}")
        print(f"Reason: {iv.get('Error', 'Doctor unavailable')}")
        print("----------------------------------")
doctors = load_doctors("doctors.json")
appointments = load_appointments("appointments.csv")
valid, invalid = validate_appointments(appointments, doctors)
generate_report(valid, invalid)
```

OUTPUT:


IDLE Shell 3.13.1
File Edit Shell Debug Options Window Help
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: D:/mine/query processing/ACTIVITY/00/02.py ==========
===== VALID APPOINTMENTS =====
Appointment ID: A001
Patient: Ram
Doctor: Dr. Meena (D101)
Date: 2024-05-01

Appointment ID: A003
Patient: Kumar
Doctor: Dr. Kiran (D103)
Date: 2024-05-03

===== INVALID APPOINTMENTS (Unavailable/Unknown Doctors) =====
Appointment ID: A002
Patient: Geetha
Doctor ID: D102
Reason: Doctor unavailable

Appointment ID: A004
Patient: Sita
Doctor ID: D999
Reason: Doctor not found

>>>